

Robust Regression Engine

How a notebook turns property details into a careful house-price estimate

*This is a no-code companion to **Robust_Reression_Engine.ipynb**. It explains the ideas behind the project, the choices made in the notebook, and what the reported numbers do and do not mean.*

3,800

property-sale
records

11

input features after
preparation

5

model families /
approaches

Who this is for

Anyone who wants to understand the project without needing to read Python. Technical words are introduced in everyday language first, then linked to the notebook term.

One important promise

A model is an evidence-based estimating tool, not a crystal ball. It learns from past examples. Its number should support a human decision, not replace judgement, local knowledge, or fairness checks.

Prepared from the notebook and included house-price CSV
All currency figures are Indian rupees (INR).

How to use this guide

A quick map

You can read from front to back, or jump straight to a question you have. The **plain-language cards** are the shortest explanation; the paragraphs underneath add the why.

Reading tip

When you see a term in **bold**, it is a notebook concept. A compact glossary appears at the end. You do not need to memorise the formulas to understand the conclusions.

- 01 The big picture: what is being predicted?
- 02 The data: what each property detail means
- 03 Preparation: turning a spreadsheet into model-ready input
- 04 Regularised linear models: Ridge and Lasso
- 05 Cross-validation: checking a model more than once
- 06 How model quality is measured
- 07 Decision trees and Random Forest
- 08 Support Vector Regression (SVR)
- 09 What the notebook results say
- 10 Limits, responsible use, glossary and FAQ

A note on the numbers

The performance figures in this guide are the outputs already recorded in the notebook. They are useful for explaining this experiment. If the data or code changes, rerun the notebook and update the report.

The notebook uses a fixed **random_state = 42** for its train/test split and some models. That is simply a repeatable starting seed: it lets another run reproduce the same random choices.

Part 1 | Start here

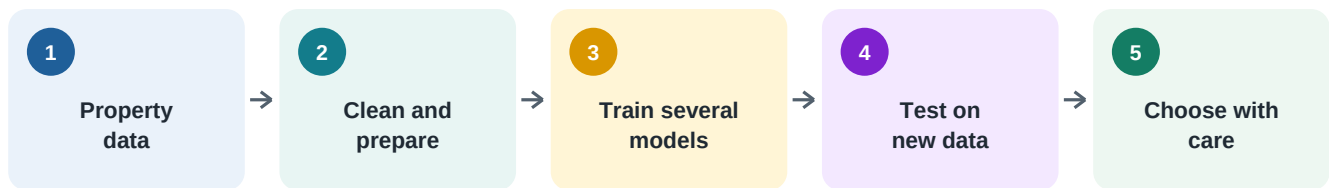
1. The big picture: a prediction problem

The project asks a practical question: **given the known details of a property, what sale price would be a reasonable estimate?** It answers this by studying thousands of past property-sale examples.

This is called **supervised machine learning**. 'Supervised' means the examples come with an answer already known - here, the recorded house price. The model looks for patterns linking property details to those answers.

Everyday analogy

Imagine asking an experienced estate agent to price a home. They would compare size, location, age and nearby amenities with past sales. The model does a similar comparison, but it learns the pattern from a table of examples rather than personal memory.



The notebook follows this loop for house-price prediction.

Regression vs classification

Regression predicts a number on a continuous scale, such as a price, temperature or travel time. **Classification** predicts a category, such as 'approved / not approved' or 'low / medium / high'. Because house price is a number, this notebook is a regression project.

A prediction is not the same as a guaranteed selling price. It is the model's best estimate based on patterns in the data it has seen. A real sale can also be affected by negotiation, property condition, market shocks and information that is not in the table.

The two roles in the data

Role	Notebook name	Plain-language meaning
Feature / input	X	A clue supplied to the model: area, bedrooms, location score, etc.
Target / output	y	The answer the model tries to learn: house_price_inr .
Prediction	y_hat	The price estimated by the fitted model for a property.
Residual / error	actual - predicted	How far the estimate missed for one sale.

The notebook creates **X** by dropping **house_price_inr** from the prepared table, then stores that price column separately as **y**. This separation matters: a model must not be allowed to peek at the answer while learning.

Part 2 | Understand the evidence

2. The data: a row is one past sale

The CSV contains **3,800** property-sale records. Each row represents one property, and each column records one detail. There are no missing values recorded in the notebook's initial check, so the workflow does not need to fill in blanks.

The source table has 12 columns. After the date is converted into useful parts and the ID is removed, the model receives 11 input features plus the price target.

Data dictionary

Source column	What it means in everyday language	How the notebook uses it
property_id	A record label for the property.	Removed. An ID is not a meaningful property characteristic.
sale_date	Date on which the sale was recorded.	Changed into sale_year and sale_month , then the original date is removed.
area_sqft	Indoor/property size in square feet.	Numeric input. Range: 500 to 3,776 sq ft.
bedrooms	Number of bedrooms.	Numeric count.
bathrooms	Number of bathrooms.	Numeric count.
location_score	A supplied rating of the location.	Numeric score from 1 to 10.
property_age	Age of the property (years).	Numeric input. Range: 1 to 80 years.
distance_city_km	Distance from the city in kilometres.	Numeric input.
near_school	Whether a school is nearby.	Binary flag: 1 means yes; 0 means no.
near_metro	Whether a metro connection is nearby.	Binary flag: 1 means yes; 0 means no.
crime_rate_index	A supplied local crime-rate index.	Numeric input. It is an index, not a direct count.
house_price_inr	Recorded sale price in Indian rupees.	The target the model tries to estimate.

Range and scale at a glance

Quantity	Observed summary from the CSV	Why it helps a reader
House price	INR 1.51M to INR 59.30M; average INR 20.72M	Shows the size of the prediction task.
Area	500 to 3,776 sq ft; average 1,716.93 sq ft	A large numerical range compared with simple counts.
Bedrooms	1 to 7; average 3.43	A small count, not a money value.
Location score	1 to 10; average 6.50	A rating with a very different scale from area.
Sale date	168 distinct month-level dates in the source	Potentially captures time-related price changes.

Important interpretation rule

A column can help prediction without being a cause. For example, a location score may stand in for many unmeasured local qualities. A model finding a pattern is not proof that changing one number will cause a price to change.

Part 3 | Prepare before predicting

3. Preparation: making the table model-ready

Models are not clever enough to fix unclear or unsuitable inputs by themselves. Data preparation makes the comparison fair and prevents accidental shortcuts. The notebook uses several common preparation steps.

3.1 Read, inspect and check

Notebook action	What it checks	Plain-language purpose
<code>pd.read_csv(...)</code>	Reads the CSV into a table called <code>df</code> .	Opens the data so it can be inspected and used.
<code>df.head()</code>	First few rows.	A quick sanity check: do columns and values look plausible?
<code>df.info()</code>	Row count, non-null counts and data types.	Checks whether a column is text, a number, or has missing entries.
<code>df.describe()</code>	Typical values, spread, smallest and largest values.	Finds surprising ranges that may signal data-entry problems.
<code>df.isnull().sum()</code>	Blank / missing values per column.	Shows where a model could otherwise fail or become biased.

3.2 Turn a date into useful clues

A date written as text (for example, 2019-12-01) is awkward for many basic models. The notebook converts `sale_date` into a date object, extracts `sale_year` and `sale_month`, and then removes the original date column. This allows a model to ask simple questions such as 'were later sales generally more expensive?' or 'is there a month pattern?'.

A useful nuance

Months wrap around: December (12) is next to January (1), not far away. A basic month number is a reasonable first step, but a mature seasonal model may encode this cycle more carefully or add richer market-time features.

3.3 Remove a record label

`property_id` is dropped. An ID usually tells us which row this is, not what the home is worth. Keeping it could let a model learn accidental ordering patterns that will not help with a genuinely new property. This is sometimes called a **spurious shortcut**.

3.4 Keep training and test data separate

The notebook uses `train_test_split(..., test_size=0.2, random_state=42)`. This creates an 80% training set (3,040 sales) and a 20% test set (760 sales). The training examples are the study material; the test examples are a final, held-back exam.

Stage	Records	What happens here
Training set	3,040 (80%)	Models learn their patterns and tuning choices are explored here.
Test set	760 (20%)	Final estimates are compared with the held-back known prices.

Why not train and test on the same rows? Because a student who memorises the answer key can look perfect on that key yet fail a new exam. A held-back test is a check on **generalisation**: whether a model can work on new, unseen examples.

3.5 Scale numerical features when a method needs it

The notebook uses **StandardScaler** for Ridge, Lasso and SVR. Scaling places numerical features onto a comparable measuring stick. Without it, an area measured in thousands of square feet could dominate a bedroom count simply because its raw numbers are larger - even when its real predictive value is not.

Standardisation for each numeric value: scaled value = (value - training average) / training standard deviation.

A scaled value of 0 means 'around the training average.' A value of +1 means 'about one typical spread above the average.' This changes the units used by the model; it does not erase the original information.

Why 'fit' only on training data?

The scaler learns the average and spread from the training rows with **fit_transform**. It then applies those same settings to the test rows with **transform**. Letting the test set influence those settings would leak future exam information into study time.

Tree-based models do not need this scaling step. A decision tree asks questions such as 'is area below 1,800?', and scaling preserves the order of values. A threshold changes its numeric label after scaling, but it divides the homes in the same order.

3.6 A Pipeline keeps steps together

For cross-validation and SVR, the notebook uses a **Pipeline**. Think of it as a sealed conveyor belt: each training fold first learns its own scaler, then fits the model. This prevents a subtle form of data leakage and makes the exact steps repeatable.

Part 4 | Linear models with guardrails

4. Regularised linear models: Ridge and Lasso

A **linear regression** model starts with a simple idea: add up the contributions of the input features. For example, it can learn that more area usually raises the estimate while greater distance from the city usually lowers it, with all other inputs held constant.

A plain-language version of the model is: **estimated price = starting price + area contribution + location contribution + age contribution + ...**. The learned multipliers are called **coefficients**. A positive coefficient raises the estimate when the feature rises; a negative coefficient lowers it.

4.1 What is a loss function?

A model needs a score that says how wrong its estimates are. Its **loss function** is that score during training. For ordinary linear regression, a common choice is the sum of squared errors: larger misses are punished more strongly. The training process searches for coefficients that make this loss small.

Error for one sale = actual price - predicted price. Squared-error loss = add up (error x error) across training sales.

4.2 Why add regularisation?

A model can become overly eager to explain every quirk in the training rows. It may assign very large, fragile coefficients that look good on old data but behave badly on new homes. **Regularisation** adds a cost for complexity, encouraging steadier coefficients. It is a guardrail, not a replacement for good data.

Everyday analogy

If several reviewers are trying to estimate a price, regularisation is a rule that says: 'Do not make an extreme claim unless the evidence is strong.' It nudges the model toward simpler, more stable explanations.

Regularisation is particularly helpful with **multicollinearity**: inputs that overlap in what they tell the model. For example, bedrooms, bathrooms and area can be related. Without a guardrail, a small data change can make the model shift a lot of credit from one related feature to another.

4.3 Ridge regression: shrink, but keep every feature

Ridge regression adds an L2 penalty: it adds the squares of the coefficients to the usual error score, multiplied by a tuning setting called **alpha**. Squaring makes especially large coefficients expensive. Ridge normally makes coefficients smaller, but does not set them exactly to zero.

Ridge objective = squared prediction error + alpha x sum of (coefficient squared).

In the notebook, Ridge is fitted after scaling. Its cross-validated best alpha is **1.0** and its test R-squared is about **0.9199**.

4.4 Lasso regression: shrink and sometimes remove

Lasso regression adds an L1 penalty: it adds the absolute sizes of the coefficients. This has a special effect: some coefficients can become exactly zero. In other words, Lasso can make a compact feature-selection decision inside the model.

Lasso objective = squared prediction error + alpha x sum of absolute coefficient sizes.

The notebook's selected Lasso alpha is **15,199.11**. It makes the **near_school** coefficient exactly zero in this fitted model.

Ridge and Lasso side by side

Question	Ridge (L2)	Lasso (L1)
Penalty	Squares of coefficients.	Absolute sizes of coefficients.
What happens to coefficients?	They are pulled closer to zero.	They are pulled closer to zero; some can become exactly zero.
Feature selection?	No: it normally retains all supplied features.	Yes: zeroed coefficients are excluded by the fitted formula.
A useful strength	Stable with overlapping / correlated features.	Produces a simpler formula when only some inputs appear useful.
A limitation	Does not itself reduce the feature list.	With correlated features, it may choose one and discard another similar clue somewhat arbitrarily.

Do not over-read a zero

Lasso setting `near_school` to zero does not mean schools have no real-world value. It means that, in this dataset, with these other features and this alpha, the model did not need that column to improve its formula enough to keep it.

4.5 Alpha: the regularisation dial

Alpha controls the strength of the penalty. Very small alpha behaves more like ordinary linear regression and can overfit. Very large alpha can flatten useful effects so much that the model **underfits**. The notebook tests 100 alpha values spread from 0.0001 to 100,000 on a logarithmic scale - a wide search that samples orders of magnitude.

Alpha values are not universal quality scores. Their best value depends on the units, target scale, model and data. The important practice is choosing alpha by cross-validation rather than by guessing from the test set.

4.6 What the fitted coefficients suggest

Because the linear-model inputs were standardised, each coefficient below is approximately the estimated INR change associated with a one-standard-deviation increase in that feature, *holding the other supplied features constant*. It is a model interpretation, not a causal claim.

Feature	Ridge coefficient	Lasso coefficient	Plain reading
area_sqft	+ INR 6.95M	+ INR 6.95M	The largest positive linear contribution in this run.
location_score	+ INR 3.68M	+ INR 3.67M	A strong positive association in the data.
property_age	- INR 0.65M	- INR 0.64M	Older homes tend to have lower estimated price after the other listed inputs are considered.
bedrooms	+ INR 0.30M	+ INR 0.29M	Small positive contribution compared with area.
near_school	+ INR 0.02M	INR 0	Retained by Ridge; removed by the selected Lasso formula.

Part 5 | Test before trusting

5. Cross-validation: several practice exams

A single training/test split is useful, but it can be lucky or unlucky depending on which records land in the test set.

Cross-validation (CV) gives a steadier estimate by repeating a train/validation process across several slices of the data.

Do not confuse the final **test set** with a CV **validation fold**. The test set is the final exam held back from model selection. Validation folds are practice exams inside the training-and-tuning process.

The core CV routine

Split the available training data into parts. Train on most parts, validate on the held-out part, rotate until every part has been held out once, then average the scores. A model that stays good across the rotations is more credible than one that is good only once.

5.1 K-Fold cross-validation

In **5-fold CV**, the data is divided into five groups. The model trains on four groups and validates on the remaining group; it repeats this five times. The notebook uses a shuffled 5-fold split with **random_state = 42**. Its Ridge pipeline has a mean validation RMSE of **INR 2.500M**.

5.2 Stratified K-Fold for a price target

Stratification tries to keep the mix of important values similar in every fold. It is straightforward for categories, but price is continuous. The notebook therefore divides price into roughly equal-frequency **bins** using **pd.qcut**, then makes each fold contain a similar mixture of low, middle and high price bands. This is a pragmatic regression technique, not a magic requirement.

The stratified 5-fold Ridge result has the lowest mean RMSE of the shown CV strategies: **INR 2.499M**. The difference from ordinary K-Fold is tiny - only about INR 111 in the displayed averages - so the key conclusion is stability, not a dramatic win.

5.3 Leave-One-Out cross-validation (LOOCV)

LOOCV uses one record as validation and all remaining records for training, repeating once per record. It makes excellent use of data, but it can be expensive. For 3,800 records, this means **3,800 separate model fits**. The notebook explains that cost rather than running the full scoring loop.

5.4 Time Series Split

When time matters, random shuffling can accidentally let a model learn from the future before predicting the past.

TimeSeriesSplit respects order: train on an earlier block and validate on a later block, then move forward. It better imitates 'predict the next period using what was known before.'

Chronology check

The notebook identifies **sale_year** and **sale_month** but applies TimeSeriesSplit to the current row order. In a real deployment, explicitly sort records by the original sale date before a time split and keep the latest period as the final test. Otherwise the rows may not be truly chronological.

What this notebook's CV results mean

Validation strategy	Mean RMSE	Plain-language reading
K-Fold (5 folds)	INR 2.500M	A stable random-slice check.
Stratified K-Fold (5 folds)	INR 2.499M	Keeps price bands balanced across the folds; almost identical to K-Fold here.
Time Series Split (5 splits)	INR 2.523M	Slightly higher error; closer to a forward-looking scenario if records are properly ordered.
LOOCV	Not run as a full score	Would require 3,800 fits, so it is less practical here.



Lower is better. Values are close, which is a reassuring sign of similar performance across these checks.

5.5 Hyperparameter tuning uses CV too

A **hyperparameter** is a setting chosen before the model is fitted, such as Ridge alpha, tree depth or SVR C. The notebook uses **RidgeCV**, **LassoCV** and **GridSearchCV** to compare candidate settings by CV score. This is safer than trying settings until the final test score looks best.

The word **negative** in scoring names such as **neg_mean_squared_error** is a library convention: scikit-learn wants higher scores to be better, so it negates an error that we naturally want to minimise. The notebook reverses the sign before taking the square root and reporting RMSE.

Part 6 | Read model scores

6. How model quality is measured

Every model predicts the same held-back test set, then its estimates are compared with the actual prices. The notebook uses four common regression metrics. No single metric tells the whole story, so reading more than one is good practice.

MAE	MSE	RMSE	R-squared
Typical miss in INR	Squares each miss	Typical miss, big errors count	Share of price variation captured
<i>Lower is better</i>	<i>Lower is better</i>	<i>Lower is better</i>	<i>Higher is better</i>

6.1 Mean Absolute Error (MAE)

MAE is the average size of the mistakes, ignoring whether the estimate was too high or too low. If MAE is INR 1.74M, think: 'on average, the prediction missed by about INR 1.74M.' It is easy to explain because it uses the same unit as the price.

MAE = average of |actual price - predicted price|.

6.2 Mean Squared Error (MSE)

MSE squares each mistake before averaging. A large miss therefore matters much more than a small miss. It is useful for training and comparison, but its unit is 'rupees squared', which is hard to explain directly. That is why RMSE is often easier to communicate.

MSE = average of (actual price - predicted price) squared.

6.3 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE. It returns to INR, while still giving extra weight to expensive mistakes. In this project, RMSE is a key comparison measure: lower means the model's errors are smaller, with a stronger penalty for large misses.

RMSE = square root of MSE.

6.4 R-squared (R2)

R-squared compares the model with a very simple baseline that always predicts the average house price. A value of 1 would be perfect. A value of 0 means no better than always guessing the average. A negative value means worse than that baseline on the test set.

R2 = 1 - (model squared error / baseline squared error).

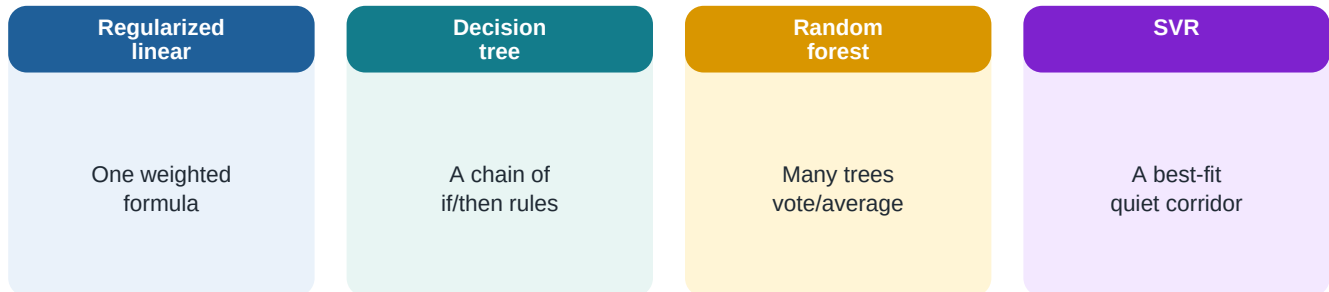
How to read the Random Forest score

Its test R2 is 0.9292. In this test set, the model captures roughly 93% of the variation in recorded prices relative to the average-price baseline. That is a strong fit for this experiment, but it is not '93% accurate' and it is not a 93% probability that any one price is correct.

Part 7 | Rule-based model families

7. Decision trees and Random Forest

The notebook also tries **tree-based** models. Instead of using one weighted formula, a tree repeatedly divides properties into groups using yes/no questions. For example: 'is area below a chosen threshold?' Then within each answer group, it asks another question. A final group (a **leaf**) makes a price estimate based on the examples inside it.



Different models look for useful patterns in different ways.

7.1 Decision Tree Regressor

A **DecisionTreeRegressor** learns thresholds that reduce the variation in price within a group. Trees handle bends and interactions naturally: perhaps area matters differently in high- and low-location-score homes. But an unrestricted tree can keep splitting until it memorises the training data, which is a classic route to overfitting.

Notebook setting	Value	Why it is a guardrail
max_depth	6	Limits how many layers of questions a tree may ask.
min_samples_split	10	Requires at least 10 examples before a group may be split.
min_samples_leaf	5	Requires at least 5 examples in a final leaf; avoids a price based on one or two unusual homes.
random_state	42	Makes any random choices repeatable.

With those limits, the single decision tree has test MAE **INR 2.115M**, RMSE **INR 2.866M** and R2 **0.8980**.

7.2 Random Forest Regressor

A **Random Forest** builds many decision trees and averages their predictions. Each tree is exposed to a slightly different view of the training data and feature choices. Averaging many imperfect but diverse trees usually reduces the tendency of a single tree to chase noise. This is an **ensemble**: a team of models rather than one model.

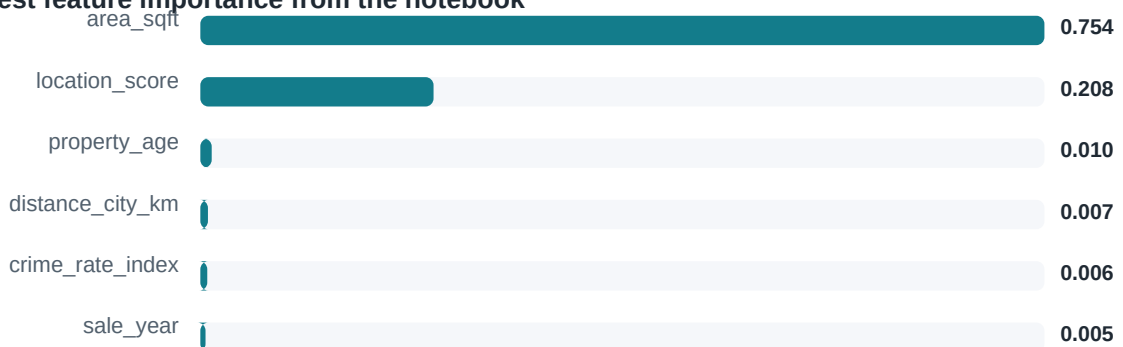
Notebook setting	Value	Plain-language meaning
n_estimators	300	Build 300 trees, then average them.
max_depth	10	Permit deeper trees than the single-tree model, but still cap complexity.
min_samples_split	5	A group needs at least 5 records before splitting.
min_samples_leaf	2	A leaf needs at least 2 records.
n_jobs	-1	Use available processor cores to build trees faster; it does not change the statistical idea.

The forest is the notebook's best test-set performer: MAE **INR 1.743M**, RMSE **INR 2.387M** and R2 **0.9292**.

7.3 Feature importance: useful, not causal

Random Forest feature importance asks how much the model's splits, on average, reduced training error using each feature. It is a model-specific clue about reliance, not a causal statement and not a guarantee that the same ordering will repeat in every sample.

Random Forest feature importance from the notebook



The remaining five inputs each have importance below 0.004. Importances add to 1 across all features.

In this run, area and location score dominate the forest's split-based importance. This makes intuitive sense for house prices, but there are important cautions:

- Importance does **not** show whether a feature raises or lowers price; it only shows how much the forest used it to split.
- Correlated clues can share or trade importance. If area and bedrooms overlap, the model may lean on one more even when both matter in reality.
- A feature that was historically useful may be unfair, outdated or unavailable at prediction time. Check appropriateness before deployment.

Why trees do not need scaling

A tree cares about order and thresholds. Whether area is written as 1,700 sq ft or as a scaled number, the same homes are above or below a proposed cut-point. Distance-based and coefficient-based methods are more sensitive to raw scale.

Part 8 | A different geometric idea

8. Support Vector Regression (SVR)

Support Vector Regression tries to draw a function through the data while ignoring small misses inside an acceptable margin. Imagine a quiet corridor around the prediction line: errors inside the corridor are treated as acceptable, and points outside it push the model to adjust. The especially influential examples are called **support vectors**.

SVR can use a **kernel**, a mathematical way to represent curved relationships without manually adding every curve as a new column. The notebook compares three kernels.

Kernel in notebook	What it can represent	Everyday analogy
linear	A straight, weighted relationship in the scaled feature space.	A flat ruler through the points.
poly (degree 2)	Polynomial curves and feature combinations of the chosen degree.	A smoothly bent ruler.
rbf	Flexible local curves based on nearby examples.	Many small zones of influence around examples.

8.1 The three main SVR controls

Control	Plain-language role	What a larger value tends to do
C	How strongly the model cares about points outside its acceptable corridor.	Penalises misses more strongly; can fit training data more tightly and overfit if pushed too far.
epsilon	Half-width of the corridor in target units (INR here).	Ignores a wider range of small errors; may produce a smoother, less sensitive fit.
gamma (RBF)	How local each example's influence is.	Makes influence more local and potentially more wiggly; too high can overfit.

The notebook starts with C=10 and epsilon=0.1 for linear and polynomial SVR, and C=100, gamma='scale', epsilon=0.1 for RBF. It correctly wraps each SVR in a scaling Pipeline because SVR is highly sensitive to feature scale.

Units matter for epsilon

The target is measured in INR, where prices are in millions. Epsilon=0.1 is tiny relative to a multi-million-rupee price. Feature scaling helps the inputs, but the notebook does not scale the target. When revisiting SVR, consider target scaling or an epsilon/C search expressed at a meaningful price scale.

8.2 GridSearchCV: an organised settings search

For RBF SVR, **GridSearchCV** tries every combination in its supplied grid: 3 values of C x 3 values of gamma x 3 values of epsilon = **27 combinations**. With 5-fold CV, it evaluates about 135 validation fits, then refits the best setting. The selected setting is C=100, epsilon=0.01, gamma='scale'.

The double underscore in names such as **svr__C** means 'the C setting of the step named svr inside the Pipeline.'

8.3 What happened here?

Every shown SVR variant is much weaker here. The tuned RBF model has RMSE **INR 8.977M** and R2 **-0.0006**, worse than the average-price baseline. This says the chosen feature representation and settings did not suit SVR; it does not make SVR a poor model in every setting.

- Potential next checks: inspect outliers, scale or transform the target, broaden C and epsilon ranges, and validate a carefully designed chronological split.
- Always compare a more complex method with a simple baseline. A complicated kernel is not automatically better.

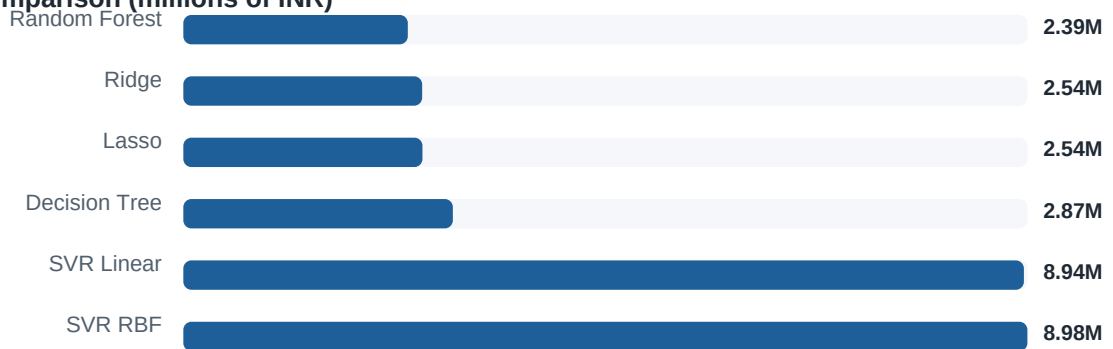
Part 9 | Bring the evidence together

9. What the experiment results say

The table below puts the recorded test-set results on one page. Lower MAE and RMSE are better; higher R2 is better. M means million INR. The models all see the same held-back test rows, which makes this comparison meaningful within this experiment.

Model	MAE	RMSE	R2	Short reading
Random Forest	INR 1.743M	INR 2.387M	0.9292	Best reported test result.
Ridge (tuned)	INR 1.945M	INR 2.540M	0.9199	Strong and stable linear baseline.
Lasso (tuned)	INR 1.947M	INR 2.543M	0.9197	Nearly Ridge-level accuracy, with one zeroed coefficient.
Decision Tree	INR 2.115M	INR 2.866M	0.8980	Useful nonlinear model, but weaker than its forest.
SVR Linear	INR 6.956M	INR 8.940M	0.0077	Very weak on this representation.
SVR RBF (tuned)	INR 6.985M	INR 8.977M	-0.0006	Worse than average-price baseline on this test.
SVR Polynomial	INR 6.994M	INR 8.987M	-0.0030	Also very weak here.

Test RMSE comparison (millions of INR)



Lower is better. SVR's large bars make the difference easy to see; do not infer a causal reason from this chart alone.

9.1 Best model in this notebook

By the reported test metrics, **Random Forest** is the best model: it has the lowest MAE and RMSE, and the highest R2. Its flexibility appears to capture non-linear relationships in these property records better than the linear formulas did.

Best does not mean automatic choice

The forest is the accuracy leader in this run, but Ridge and Lasso are easier to explain feature-by-feature and show a much smaller train/test gap. The right production choice depends on the cost of error, the need for explanation, stability over time, speed, and fairness - not only one leaderboard score.

9.2 Train versus test: spotting overfitting and underfitting

Overfitting means a model learns peculiarities of its training examples and then drops noticeably on new examples.

Underfitting means it is too simple, too constrained or poorly configured to learn the important pattern even in training.

The **generalisation gap** is test RMSE minus train RMSE; a large positive gap is a warning sign.

Model	Train RMSE	Test RMSE	Gap	Notebook interpretation
Random Forest	INR 1.344M	INR 2.387M	INR 1.044M	Some overfitting, despite best test score.
Ridge	INR 2.482M	INR 2.540M	INR 0.058M	Very stable generalisation.
Lasso	INR 2.483M	INR 2.543M	INR 0.060M	Very stable generalisation.

Model	Train RMSE	Test RMSE	Gap	Notebook interpretation
Decision Tree	INR 2.437M	INR 2.866M	INR 0.429M	Moderate overfitting.
SVR RBF	INR 8.657M	INR 8.977M	INR 0.320M	High error on both: underfitting / poor fit.

The gap is a diagnostic, not a final verdict. A low train error is not impressive if the test error is poor; a modest gap is desirable only when the test error itself is also acceptable.

9.3 A sensible decision path

- Use the **Random Forest** as the current accuracy benchmark, then validate it on newer, truly chronological data before deployment.
- Keep **Ridge or Lasso** as transparent reference models. When a forest estimate seems surprising, a simple model can help explain broad drivers.
- Do not deploy SVR from this notebook as-is; it is not competitive in the recorded results. Revisit it only with a justified redesign and validation plan.
- Report uncertainty and a price range, not just a single point estimate. The current notebook evaluates average error but does not yet calculate prediction intervals.

Part 10 | Use the result responsibly

10. Limits, responsible use and next steps

A good model evaluation is not the end of the work. Before anyone relies on a house-price estimate, check whether the data, process and use case are appropriate.

10.1 What this notebook does not establish

- **Causation:** The model detects patterns in recorded sales; it does not prove that changing one feature causes the price change the model estimates.
- **Future market certainty:** A train/test split measures performance on held-back historical rows, not on an unknown future market shock.
- **All property details:** The table has no visible information about condition, renovations, exact neighbourhood, legal status, floor level, views or negotiation context. Missing information limits any estimate.
- **Fairness:** Location-linked variables can reflect historic inequalities. Review inputs and outcomes by relevant groups and use the tool as decision support, not an unchallengeable authority.
- **Individual guarantee:** An average error such as MAE does not say the error for one particular home will be that exact amount.

10.2 Practical improvements for a future version

Improvement	Why it matters
Chronological holdout	Sort by sale date; train on earlier sales and test on the latest period to mirror real forecasting.
Data-quality review	Check duplicate properties, outliers, changing definitions and whether input values would be known at prediction time.
Uncertainty intervals	Provide a plausible range around a price, not only a point estimate.
Feature engineering	Consider price per area patterns, non-linear age effects, local market indicators and well-justified date seasonality.
Explainability checks	Use local explanations and permutation/holdout checks; avoid treating one feature-importance chart as proof.
Monitoring	After deployment, compare estimates with later sales and watch for performance drift.

A healthy production question

Ask: 'Would we have known every input at the moment we make this prediction, and would we be comfortable explaining this estimate to the person affected by it?' If the answer is no, improve the process before use.

10.3 The key takeaway

The notebook provides a thoughtful model comparison for a house-price task. In the recorded run, Random Forest has the strongest test metrics, while Ridge and Lasso provide highly competitive, more transparent alternatives. The most important skill is not memorising model names; it is keeping data preparation, validation, interpretation and responsible use connected.

Plain-language glossary

These short definitions are designed for quick reference while reading the notebook.

Alpha	The regularisation strength in Ridge and Lasso. It is a complexity dial chosen by validation.
Baseline	A simple reference prediction. Here, the average price is a baseline for R-squared.
Coefficient	A learned weight in a linear model that changes the estimated price when an input changes.
Cross-validation	Repeated train/validate rotations used to check a model and select settings.
Data leakage	Using information during training that would not be available when making a real future prediction.
Feature	An input clue supplied to a model, such as area or bedrooms.
Generalisation	How well a model works on new data that it did not use to learn.
Hyperparameter	A setting chosen before fitting, such as tree depth, alpha, C or gamma.
Kernel	An SVR mechanism for representing relationships that may not be straight lines.
Lasso / L1	A regularised linear model that can shrink some coefficients exactly to zero.
Loss	The numeric score a training process tries to minimise; lower means the fit is better by that definition.
MAE	Average absolute miss, expressed in the original unit (INR).
Model	A learned rule that turns inputs into an estimated output.
Overfitting	Learning the training examples too specifically, then performing worse on new data.
Pipeline	A bundled sequence of repeatable steps, such as scaling followed by model fitting.
R2 / R-squared	A comparison with an average-only baseline; 1 is perfect, 0 matches baseline, negative is worse.
Random Forest	An ensemble that averages many decision trees.
Regularisation	A penalty that discourages overly complicated or extreme model behaviour.
Residual	The difference between a real value and the model's prediction.
RMSE	Square-rooted average squared miss; in INR and more sensitive to large errors than MAE.
Ridge / L2	A regularised linear model that shrinks coefficients while usually keeping every feature.
Scaling	Putting numeric inputs on comparable ranges; StandardScaler uses the training average and spread.
Target	The answer a supervised model is trying to estimate; here, house_price_inr.
Underfitting	Failing to learn useful patterns, resulting in poor performance even on training data.
Validation fold	The held-out part of one cross-validation rotation, used for a practice evaluation.

Close with confidence

Frequently asked questions

Can I say the model is 93% accurate?

No. The best R^2 is 0.9292, which describes how much of the test-set price variation the model captures relative to an average-price baseline. It is not a percentage of individual prices that are correct.

Why is the best model still wrong by millions of INR?

House prices themselves range from about INR 1.51M to INR 59.30M, and the data does not include every influence on a sale. An average error also hides a spread: some estimates will be much closer and some much farther away.

Why keep Ridge or Lasso if Random Forest wins?

They are strong benchmarks with simple, inspectable coefficients and much smaller train/test gaps. A small accuracy trade-off may be worthwhile when explanation, auditability or stability matters.

Does a high feature importance prove what causes price?

No. It only reports how the fitted forest used a feature to reduce error on this data. Correlation, omitted details and related variables mean causal claims need a different study design.

Why use a test set and cross-validation both?

Cross-validation helps choose models and settings without leaning on one lucky split. The held-back test is the final independent check after those choices.

What should happen before use on current listings?

Validate on recent chronologically later sales, audit inputs and fairness, decide an acceptable error range, explain uncertainty to users, and monitor performance after release.

Notebook-to-guide traceability

This guide is a theory companion, not a replacement for the notebook. The following map helps readers connect explanations back to the code.

Notebook area	Guide explanation
Part A	Regularisation, Ridge vs Lasso, CV basics, and why tree models do not need scaling.
Part B	Dataset dictionary, date transformation, ID removal, feature/target split, train/test split and scaling.
Part C	Linear loss, Ridge/Lasso penalties, alpha search, fitted coefficients and feature selection.
Part D	K-Fold, stratified price bins, LOOCV cost, Time Series Split and CV results.
Part E	Decision Tree controls, Random Forest ensemble and feature importance cautions.
Part F	SVR kernels, C/gamma/epsilon, Pipeline and GridSearchCV.
Parts G-H	MAE/MSE/RMSE/ R^2 , model comparison, train/test gaps and final conclusions.

Final thought

A useful prediction system is not just an algorithm with a high score. It is clear about its evidence, honest about uncertainty, tested on the right future-like data, and used with human judgement.

End of guide